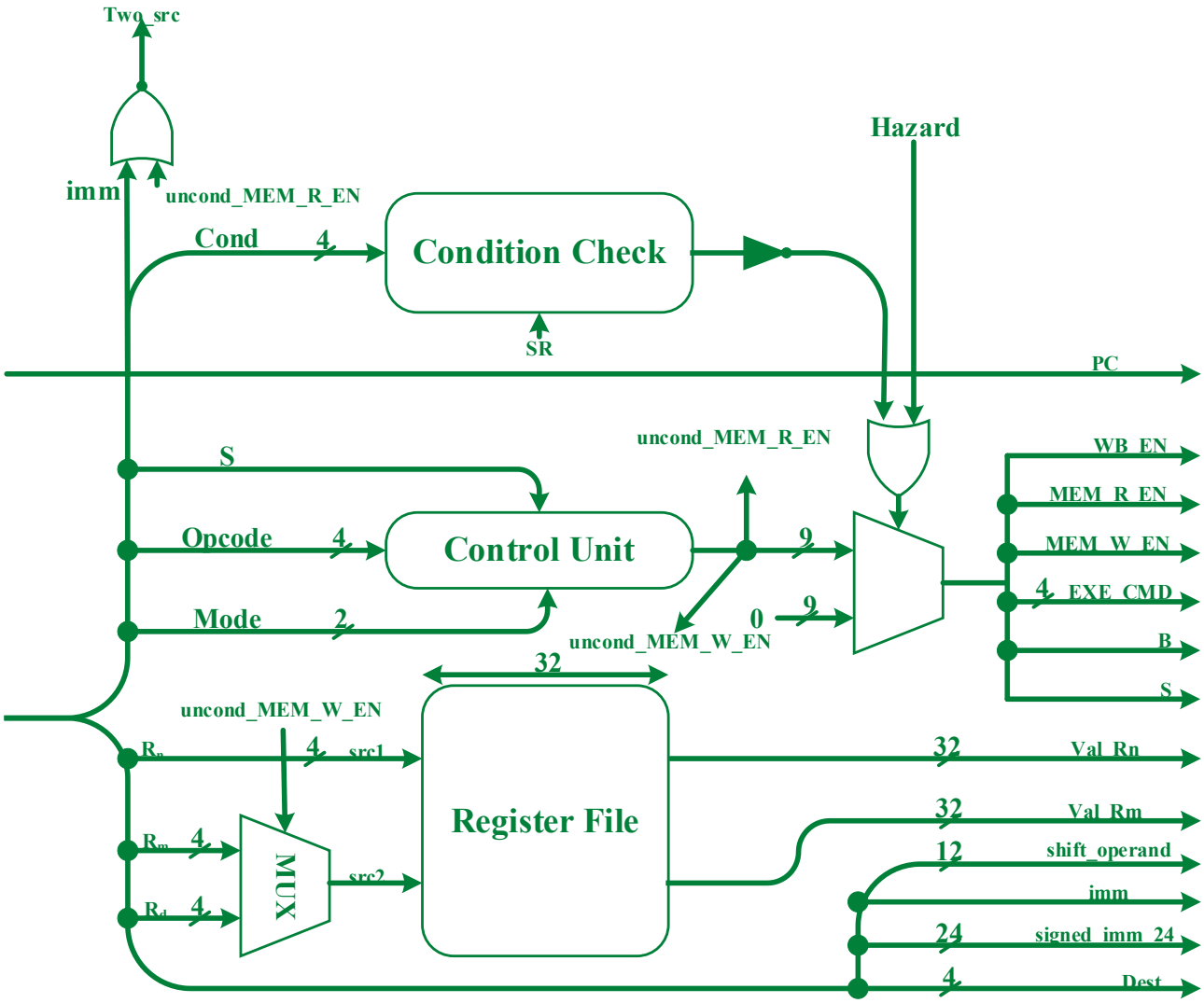
قسمت دوم آزمایش ARM: (جلسه دوم)

در جلسه پیش مرحله واکشی دستور را پیاده نموده‌اید در این جلسه مرحله کدگشایی را پیاده خواهید کرد.



برای این کار به روش زیر عمل نمایید:

۱- ماژول ثبات‌های عمومی (Registers File) را با پورت‌های زیر ایجاد نمایید. برای پیاده‌سازی آن از یک آرایه دوبعدی استفاده نمایید.

```
module RegisterFile(
    input clk,
    input rst,
    input [3:0] src_1,
    input [3:0] src_2,
    input [3:0] Dest_WB,
    input [31:0] Result_WB,
    input writeBackEN,
    output [31:0] reg_out_1,
    output [31:0] reg_out_2
);
```

۲- ماژول ثبات‌های عمومی قابلیت خواندن همزمان از ۲ ثبات به صورت ناهمگام و نوشتن در رجیستر به صورت همگام (با لبه پایین رونده) را دارد.

۳- ماژول ثبات‌های عمومی دارای ۱۶ ثبات ۳۲بیتی است. ثبات ۱۵ آن خاص منظوره بوده و به عنوان شمارنده دستورات مورد استفاده قرار می گیرد برای سادگی کار از ماژول ثبات های عمومی خارج شده است بنابراین ماژول ثبات های عمومی را با ۱۵ رجیستر پیاده‌سازی کنید. همه ثبات‌ها را مطابق با شماره آن مقداردهی اولیه نمایید.

٤- ماژول Control Unit را ایجاد نمایید. این ماژول دو بیت mode و ۴ بیت Op-code به همراه بیت S را دریافت می‌کند و سیگنال‌های خروجی شامل این موارد است:

الف) Execute Command (ریز دستورهای واحد حساب و منطق مطابق **جدول ۵ از دستور کار ARM**).

ب) سیگنال‌های مرحله حافظه شامل خواندن از حافظه mem_read و نوشتن در حافظه mem_write.

ج) سیگنال مربوط به فعال سازی مرحله باز نشانی WB_Enable.

د) سیگنال مربوط به دستورات Immediate بودن یا نبودن (Imm).

هـ) سیگنال مربوط به دستور پرش B

و) سیگنال مربوط به به روزرسانی ثبات وضعیت. این سیگنال در تمامی دستورات محسابی منطقی(mode=0) برابر ورودی S خواهد بود.

توجه: در صورت استفاده مستقیم از کد دستور(op-code) در هر ماژولی بجز Control Unit، نمره کسر می گردد.

۵- ماژول condition_Check را تعریف کنید. این ماژول بیت‌های مربوط به شرط (cond) را از دستور ورودی را دریافت می‌کند و به کمک ثبات وضعیت برقرار بودن شرط را بررسی می‌کند. بیت‌های مربوط به رجیستر SR از ورودی مرحله کدگشایی دریافت می‌شود. **شرایط برقرار بودن شرط در جدول ۳ از ARM ذکر شده است.**

٦- ماژول کدگشایی و رجیسترهای پس از آن را مطابق با ورودی‌های مشخص شده در دستورکار ARM ایجاد نمایید.

٧- از ماژول ثبات‌های عمومی (RegisterFile) در ماژول ID_Stage نمونه‌گیری (instantiate) کرده و پایه‌های آن را متصل نمایید.

- اتصال پورت src2: پورت src2 (آدرس دوم خواندن) نیازمند یک MUX (انتخاب‌گر) است. سلکتور این MUX، سیگنال uncond_mem_write می‌باشد:
  - دستور STR (uncond_mem_write=1): پورت src2 باید به فیلد رجیستر Dest (همان Rd در دستور STR) متصل شود. (زیرا این رجیستر، حاوی داده‌ای است که باید خوانده شده و در حافظه نوشته شود).
  - سایر دستورات (uncond_mem_write=0): پورت src2 باید به فیلد رجیستر Rm متصل شود.

❖ توضیح سیگنال _uncond : منظور از پسوند _uncond (مخفف Unconditional) در سیگنال‌های کنترلی، خروجی‌های خام و مستقیم از Control Unit است؛ یعنی قبل از آنکه این سیگنال‌ها توسط منطق بررسی شرط (Cond_out) یا منطق رفع خطر (Hazard) فیلتر یا متوقف (Gated) شوند.

٨- از ماژول control_unit در مرحله کدگشایی نمونه گیری کنید.

٩- از ماژول condition_Check در مرحله کدگشایی نمونه گیری کنید. در صورت برقرار نبودن شرط دستور تمامی خروجی های control_unit صفر می شود بنابراین یک mux بعد از خروجی این ماژول قرار می گیرد. Selector آن خروجی condition_Check خواهد بود.

١٠- سیگنال های خروجی مرحله کدگشایی را متصل کنید.

١١- از ماژول کدگشایی و رجیستر بعد از آن در ماژول اصلی(Top Module) نمونه‌گیری کنید.

١٢- سیگنال‌هایی که هنوز پیاده‌سازی نشده اند را به صفر متصل کنید.

١٣- تمامی سیگنال‌های کنترلی خروجی مرحله کدگشایی و رجیسترهای بعد از آن به همراه رجیسترهای Rn و Rm را در ILA(Integrated Logic Analyzer) مانند شکل ٢ نمایش دهید.

١٤- درستی سیگنال های تولید شده را بررسی نمایید. توجه داشته باشید که این سیگنال ها باید با دستورات مطابقت داشته باشد به طور مثال سیگنال mem_read فقط در دستور LDR باید برابر یک شود.

١٥- مقادیر رجیسترهای Rn و Rm باید مطابق عدد رجیستر باشد زیرا براساس عدد آن مقداردهی اولیه شده است.

١٦- در صورت درستی، نتایج را به دستیار آموزشی نشان دهید.


Name
clk
rst
B
S
imm
WB_EN
Two_src
MEM_W_EN
MEM_R_EN
Dest[3:0]
EXE_CMD[3:0]
src_1[3:0]
src_2[3:0]
Shift_operand[11:0]
Signed_imm_24[23:0]
Val_Rn[31:0]
Val_Rm[31:0]

شکل ۲